

5-7 LPA Placement Preparation

Questions 31-40 — C++

18 pt title • 16 pt question headings • 14 pt body & code • 1.5 line spacing • Colorful code blocks

31. Find Duplicate Elements in an Array

Example: [1, 2, 3, 2, 4, 1] → Duplicate elements: 1 2

Brute Force — compare every pair:

```
int arr[] = {1, 2, 3, 2, 4, 1};
```

```
int n = 6;
```

```
for(int i = 0; i < n; i++) {  
    bool alreadyPrinted = false;
```

```
    for(int k = 0; k < i; k++) {  
        if(arr[k] == arr[i]) {  
            alreadyPrinted = true;  
            break;  
        }  
    }
```

```
    if(alreadyPrinted)  
        continue;
```

```
    for(int j = i + 1; j < n; j++) {  
        if(arr[i] == arr[j]) {  
            cout << arr[i] << " ";  
            break;  
        }  
    }  
}
```

Complexity: Time $O(n^2)$, Extra Space $O(1)$.

Optimized — frequency array for a known small positive range:

```
int arr[] = {1, 2, 3, 2, 4, 1};
```

```
int n = 6;
int freq[101] = {0};
```

```
for(int i = 0; i < n; i++)
    freq[arr[i]]++;
```

```
for(int i = 0; i < 101; i++)
    if(freq[i] > 1)
        cout << i << " ";
```

Complexity: Time $O(n + \text{range})$, Extra Space $O(\text{range})$.

For arbitrary values, an unordered_set/frequency map is preferable.

32. Find Missing Number in an Array

Example: [1, 2, 3, 5] → Missing = 4

Assumption: Array contains numbers from 1 to n, with exactly one missing number.

Brute Force — check every number:

```
int arr[] = {1, 2, 3, 5};
int n = 5;
```

```
for(int num = 1; num <= n; num++) {
    bool found = false;
```

```
    for(int i = 0; i < n - 1; i++) {
        if(arr[i] == num) {
            found = true;
            break;
        }
    }
```

```
    if(!found) {
        cout << "Missing = " << num;
        break;
    }
```

```
}
```

Complexity: Time $O(n^2)$, Extra Space $O(1)$.

Optimized — Sum formula:

```
long long expected = 1LL * n * (n + 1) / 2;
```

```
long long actual = 0;
```

```
for(int i = 0; i < n - 1; i++)  
    actual += arr[i];
```

```
cout << "Missing = " << expected - actual;
```

Complexity: Time $O(n)$, Extra Space $O(1)$.

Important: XOR can also solve this in $O(n)$ without overflow.

33. Find Frequency of Array Elements

Example: [1, 2, 2, 3, 1, 2] → 1=2, 2=3, 3=1

Brute Force:

```
int arr[] = {1, 2, 2, 3, 1, 2};
```

```
int n = 6;
```

```
for(int i = 0; i < n; i++) {  
    bool already = false;
```

```
    for(int j = 0; j < i; j++) {  
        if(arr[i] == arr[j]) {  
            already = true;  
            break;  
        }  
    }  
}
```

```
if(already)  
    continue;
```

```
int count = 0;
```

```

for(int j = 0; j < n; j++) {
    if(arr[i] == arr[j])
        count++;
}

cout << arr[i] << " -> " << count << endl;
}

```

Complexity: Time $O(n^2)$, Extra Space $O(1)$.

Optimized — Frequency array:

int freq[101] = {0};

```

for(int i = 0; i < n; i++)
    freq[arr[i]]++;

for(int i = 0; i < 101; i++) {
    if(freq[i] > 0)
        cout << i << " -> " << freq[i] << endl;
}

```

Complexity: Time $O(n + \text{range})$, Extra Space $O(\text{range})$.

For arbitrary integers, use `unordered_map<int,int>`.

34. Find Common Elements in Two Arrays

Example:

A = [1, 2, 3, 4]

B = [2, 4, 6, 8]

Common = 2 4

Brute Force:

int a[] = {1, 2, 3, 4};

int b[] = {2, 4, 6, 8};

int n = 4, m = 4;

```

for(int i = 0; i < n; i++) {
    bool found = false;

```

```
for(int j = 0; j < m; j++) {  
    if(a[i] == b[j]) {  
        found = true;  
        break;  
    }  
}
```

```
if(found) {  
    bool printed = false;
```

```
    for(int k = 0; k < i; k++)  
        if(a[k] == a[i])  
            printed = true;
```

```
    if(!printed)  
        cout << a[i] << " ";  
}
```

Complexity: Time $O(n*m)$, Extra Space $O(1)$.

Optimized — Mark elements of the first array:

int freq[101] = {0};

```
for(int i = 0; i < n; i++)  
    freq[a[i]] = 1;
```

```
for(int j = 0; j < m; j++) {  
    if(freq[b[j]] == 1) {  
        cout << b[j] << " ";  
        freq[b[j]] = 2;  
    }  
}
```

Complexity: Time $O(n + m + \text{range})$, Extra Space $O(\text{range})$.

35. Merge Two Sorted Arrays

Example:

A = [1, 3, 5]

B = [2, 4, 6]

Merged = [1, 2, 3, 4, 5, 6]

Brute Force — copy and manually sort:

int a[] = {1, 3, 5};

int b[] = {2, 4, 6};

int n = 3, m = 3;

```
int c[6];
```

```
for(int i = 0; i < n; i++)  
c[i] = a[i];
```

```
for(int i = 0; i < m; i++)  
c[n + i] = b[i];
```

```
for(int i = 0; i < n + m; i++) {  
    for(int j = i + 1; j < n + m; j++) {  
        if(c[i] > c[j]) {  
            int temp = c[i];  
            c[i] = c[j];  
            c[j] = temp;  
        }  
    }  
}
```

```
for(int i = 0; i < n + m; i++)  
cout << c[i] << " ";
```

Complexity: Time $O((n+m)^2)$, Extra Space $O(n+m)$.

Optimized — Two Pointer:

int c[6];

int i = 0, j = 0, k = 0;

```
while(i < n && j < m) {  
    if(a[i] <= b[j])  
        c[k++] = a[i++];  
    else
```

```
c[k++] = b[j++];  
}
```

```
while(i < n)  
c[k++] = a[i++];
```

```
while(j < m)  
c[k++] = b[j++];
```

```
for(int x = 0; x < n + m; x++)  
cout << c[x] << " ";
```

Complexity: Time $O(n+m)$, Extra Space $O(n+m)$.

36. Find Intersection of Two Arrays

Example:

A = [1, 2, 2, 3]

B = [2, 2, 4]

Intersection with duplicate occurrences = [2, 2]

Brute Force — match and mark:

```
int a[] = {1, 2, 2, 3};
```

```
int b[] = {2, 2, 4};
```

```
int n = 4, m = 3;
```

```
bool used[3] = {false};
```

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < m; j++) {  
        if(a[i] == b[j] && !used[j]) {  
            cout << a[i] << " ";  
            used[j] = true;  
            break;  
        }  
    }  
}
```

Complexity: Time $O(n*m)$, Extra Space $O(m)$.

Optimized — Frequency counting:

```
int freq[101] = {0};
```

```
for(int i = 0; i < n; i++)  
    freq[a[i]]++;
```

```
for(int j = 0; j < m; j++) {  
    if(freq[b[j]] > 0) {  
        cout << b[j] << " ";  
        freq[b[j]]--;  
    }  
}
```

Complexity: Time $O(n+m+\text{range})$, Extra Space $O(\text{range})$.

Note: If the interviewer means unique intersection only, print each value once.

37. Find Union of Two Arrays

Example:

A = [1, 2, 3]

B = [2, 3, 4]

Union = 1 2 3 4

Brute Force — build result manually:

```
int a[] = {1, 2, 3};
```

```
int b[] = {2, 3, 4};
```

```
int n = 3, m = 3;
```

```
int result[6];
```

```
int k = 0;
```

```
for(int i = 0; i < n; i++) {  
    bool found = false;
```

```
    for(int j = 0; j < k; j++)  
        if(result[j] == a[i])  
            found = true;
```



```
if(!found)
result[k++] = a[i];
}
```

```
for(int i = 0; i < m; i++) {
bool found = false;
```

```
for(int j = 0; j < k; j++)
if(result[j] == b[i])
found = true;
```

```
if(!found)
result[k++] = b[i];
}
```

```
for(int i = 0; i < k; i++)
cout << result[i] << " ";
```

Complexity: $O((n+m)^2)$, Extra Space $O(n+m)$.

Optimized — Frequency/visited array:

bool seen[101] = {false};

```
for(int i = 0; i < n; i++) {
if(!seen[a[i]]) {
cout << a[i] << " ";
seen[a[i]] = true;
}
}
```

```
for(int i = 0; i < m; i++) {
if(!seen[b[i]]) {
cout << b[i] << " ";
seen[b[i]] = true;
}
}
```

Complexity: Time $O(n+m+range)$, Extra Space $O(range)$.

38. Move All Zeros to the End

Example: [0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]

Brute Force — extra array:

```
int arr[] = {0, 1, 0, 3, 12};
```

```
int n = 5;
```

```
int temp[5];
```

```
int k = 0;
```

```
for(int i = 0; i < n; i++)  
if(arr[i] != 0)  
temp[k++] = arr[i];
```

```
while(k < n)  
temp[k++] = 0;
```

```
for(int i = 0; i < n; i++)  
cout << temp[i] << " ";
```

Complexity: Time $O(n)$, Extra Space $O(n)$.

Optimized — Two pointer:

```
int j = 0;
```

```
for(int i = 0; i < n; i++) {  
if(arr[i] != 0) {  
int temp = arr[i];  
arr[i] = arr[j];  
arr[j] = temp;  
j++;  
}  
}
```

```
for(int i = 0; i < n; i++)  
cout << arr[i] << " ";
```

Complexity: Time $O(n)$, Extra Space $O(1)$.

Important: Relative order of non-zero elements remains unchanged.

39. Find Maximum and Minimum Difference

Interpretation: Find the difference between the maximum and minimum array elements.

Example: [10, 4, 8, 2, 15] → Max-Min = 15-2 = 13

Brute Force:

int maxVal = arr[0];

int minVal = arr[0];

```
for(int i = 0; i < n; i++) {  
    for(int j = i + 1; j < n; j++) {  
        if(arr[i] > arr[j]) {  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
}
```

```
cout << "Difference = " << arr[n - 1] - arr[0];
```

Complexity: Time $O(n^2)$, Extra Space $O(1)$, but this modifies the array.

Optimized — Single Scan:

int maximum = arr[0];

int minimum = arr[0];

```
for(int i = 1; i < n; i++) {  
    if(arr[i] > maximum)  
        maximum = arr[i];
```

```
    if(arr[i] < minimum)  
        minimum = arr[i];  
}
```

```
cout << "Difference = " << maximum - minimum;
```

Complexity: Time $O(n)$, Extra Space $O(1)$.

Interview tip: If they ask "maximum difference arr[j]-arr[i] with j>i", that is a different problem and needs a different approach.

40. Find Pair with Given Sum

Example:

Array = [2, 7, 11, 15], Target = 9

Pair = 2, 7

Brute Force — check every pair:

```
int arr[] = {2, 7, 11, 15};
```

```
int n = 4;
```

```
int target = 9;
```

```
bool found = false;
```

```
for(int i = 0; i < n; i++) {  
    for(int j = i + 1; j < n; j++) {  
        if(arr[i] + arr[j] == target) {  
            cout << arr[i] << " + " << arr[j] << " = " << target <<  
            endl;  
            found = true;  
        }  
    }  
}
```

```
if(!found)  
    cout << "No pair found";
```

Complexity: Time $O(n^2)$, Extra Space $O(1)$.

Optimized — Hashing:

```
int arr[] = {2, 7, 11, 15};
```

```
int n = 4;
```

```
int target = 9;
```

```
unordered_set<int> seen;
```

```
for(int i = 0; i < n; i++) {  
    int needed = target - arr[i];
```

```
if(seen.find(needed) != seen.end()) {  
    cout << needed << " + " << arr[i]  
    << " = " << target;  
    break;  
}
```

```
seen.insert(arr[i]);  
}
```

Average Complexity: Time $O(n)$, Extra Space $O(n)$.

If STL/hash table is not allowed, for a sorted array use the two-pointer method in $O(n)$.

□ Quick Interview Revision

Q31-34 → Frequency / duplicate / common-element patterns

Q35-37 → Two-pointer + union/intersection patterns

Q38 → Two-pointer / in-place array modification

Q39 → Single-pass min/max

Q40 → Brute-force pair checking → Hashing optimization